AI 开发全景入门

AI Coding Agent 实战工作坊 — 初阶场 (Codex)

目标： 了解 AI 辅助开发的完整生态，并且自己成功完成第一次 AI 协作

John Wu | Apr 2026

今天的旅程

时段	做什么
观念讲解	先建立基本观念
Live Demo	看 AI 怎么一步步做事
练习时间	套到你自己的工作场景
回顾 Q&A	重点整理 + 课后练习说明

> 今天的重点是 **体验一次完整流程**，不是把每个名词背起来

Part 1: 观念讲解

什么是 Vibe Coding？

传统方式

你 → 想逻辑 → 写 code → debug → 完成

Vibe Coding

你 → **描述意图** → AI 生成 → **你验收** → 完成

> 不再逐行手写，而是用自然语言「说出你要什么」

三个心智模型转换

以前	现在
搜索 — Google, Stack Overflow	**对话** — 直接问 AI，它给你完整答案
复制粘贴 — 找到 code 改改看能不能用	**描述意图** — 说你要什么，AI 量身打造
独自开发 — 自己想、自己写	**人机协作** — 你做决策，AI 做执行

AI 工具

	Claude Code	Codex	Gemini CLI
公司	Anthropic	OpenAI	Google
项目指引文件	CLAUDE.md	AGENTS.md	GEMINI.md
延伸能力	Skills / MCP	Skills / MCP	Extensions / MCP
共通概念	都能有项目指引文件，也都能接外部工具与数据来源		

> 今天初阶以 **Codex** 为主要工具

Prompts — 你当下怎么开口

白话

你对 AI 说的每一句话，都是 Prompt

示例

「帮我在 `src/bookmark.ts` 加一个导出功能」

> 描述越具体，AI 的结果越接近你要的

Project Instructions — 让 AI 先了解你的项目

白话

在项目放一个项目指引文件，AI 自动读取，不用每次重讲

示例

`AGENTS.md`（Codex）里面写：
「这是 TypeScript 项目，用 Vitest 测试，commit 格式是 type(scope): description」

> 写一次，全团队受益 — AI 每次开工都知道规矩

Skills — 让 AI 按照流程做事

白话

把常见工作流程写成可重用的能力，让 AI 在需要时套用

示例

写好一个「TDD skill」后：

- 你可以明确指定它使用这个 skill
- 或在任务描述吻合时，让 AI 自动套用

> 不用每次重复交代，写一次就能重复使用

MCP — 让 AI 连接外部工具与数据

白话

让 AI 不只看你贴的内容，还能连到外部系统查数据、读文档、做操作

示例

- 连 GitLab → 读 Issue、MR
- 连 JIRA → 读 Ticket
- 连 Telegram → 收发消息

> AI 从「只会聊天」进一步变成「能查、能读、能动手做事」

用「开餐厅」来比喻

你说的话	餐厅比喻	对应概念
「我想吃辣的」	先做一版试吃再调整	Vibe Coding
「宫保鸡丁，不要花生，少油」	讲越具体，出餐越准	Prompts
「这是川菜馆，大火快炒」	厨房规矩写好	Project Instructions
食谱 + 检查点 + 工具清单	标准作业流程	Skill
接上供应商、仓库、外送系统	对外接口	MCP

Context — 这些东西怎么运作的？

每次你跟 AI 对话，工具通常会整理相关的 **Context** 给它，例如：

```
你的 prompt + 对话历史 + 相关文件内容 + 项目指引文件（如 AGENTS.md）+ 需要时加载的 Skills + 调用 MCP 后获取的结果
```

代价

- Context 越大 → 通常越慢、越贵
- 对话越长 → 前面不够突出的重点，越可能被稀释
- 带太多不相关信息 → 判断质量会下降

> 所以：讲重点、必要时开新对话、不要把什么都塞进去

Models — AI 的法术强度

你可以先把不同模型想成游戏里不同等级的法术：

法术等级	感觉	代价	适合什么时候用
小招级	最快、最省	🔥	整理文案、格式修正、简单问答
主力技	速度和质量平衡	🔥🔥	日常 feature 开发、大部分工作
大招级	处理难题比较稳	🔥🔥🔥	卡很久的 bug、架构设计、复杂重构

> 不同工具可选的模型名称会变，但挑选原则差不多：
> **简单事别乱开大招，杀鸡不用牛刀**

推理程度 — 法术蓄力

同一个法术，也可以想成有不同蓄力时间：

蓄力	感觉	代价	适合什么时候用
低	几乎秒放	🔥	很简单的修改、格式整理
中	想一下再出手	🔥🔥	一般开发任务
高	想比较久，出手比较重	🔥🔥🔥	复杂逻辑、多步骤问题
超高	全力蓄力再丢大招	🔥🔥🔥🔥	困难 debug、架构设计、难拆的大任务

> 蓄力越久，通常答案会更完整；
> 但也会更慢、更贵。所以不是每次都要拉满。

Token — AI 的魔力值

把 **Token** 想成 AI 处理信息时的计量单位，也可以先理解成「魔力消耗单位」。

- 你讲得越多，通常越耗
- AI 回得越多，通常越耗
- 对话越长、内容越乱，通常越耗

> 所以：讲重点，少废话，卡住就开新对话

魔力用完怎么办？

- 不同工具、不同方案，限制单位不太一样
- 有些看 token，有些看消息次数、使用额度或速率限制
- 有些方案会定期恢复
- 有些方案可以升级或加购，但不是每种都可以
- 强模型、长对话、重工具使用，通常都更耗资源

> 所以：简单事用小招，难题再开大招
> 具体限制与恢复方式，还是要看工具的官方方案说明

初阶你只要记得三件事

1. 先把需求讲清楚
2. 再让 AI 动手做
3. 最后一定要验收结果

> 今天的成功标准不是懂所有工具，而是完成一次可验收的协作

Part 2: Live Demo

Codex 界面介绍

新增照片和檔案
規劃模式
Speed
GPT-5.4
本機
Standard
Default speed with normal credit usage
Fast
About 1.5x faster, with credits used at 2x
繼續
本機專案
剩餘的速率限制
5 小時 100% 凌晨12:51
1 週 94% 4月14日
如需取得更多存取權，請聯絡你的管理員。
深入了解
本機
預設權限
新增對話串
Search
技能和應用程式
自動化
對話串
live-demo
沒有對話串
database
terraform
handbook
sandbox-stack
設定
新增對話串
讓我們開始建立吧
live-demo
Build a classic Snake game in this repo.
Create a one-page $pdf that summarizes this app.
Create a plan to...
Ask Codex anything, @ to add files, / for commands, $ for skills
GPT-5.4
中
本機
預設權限
選取模型
GPT-5.4
GPT-5.4-Mini
GPT-5.3-Codex
GPT-5.2
選取推理
低
中
高
超高
上下文視窗：
已使用 14%（剩餘 86%）
已使用 35k / 258k 個自字元
Codex 會自動壓縮上下文

Demo 1 — Vibe Coding

一句话做出一个游戏

1. 跟 Codex 说：**「帮我做一个打地鼠游戏」**
2. 看到第一版后，给反馈迭代：
   - 「打到地鼠时加爆炸特效」
   - 「地鼠出现速度要越来越快」
3. 满意后收工

Demo 2 — Prompts 的差异

同一个需求，不同的说法

	Prompt	预期结果
❌	「改好一点」	AI 不知道要改什么
⚠️	「UI 丑丑的，帮我改」	AI 随便改，可能不是你要的
✅	「把背景改成草地主题，地鼠换成卡通风格，按钮加圆角和阴影，被打到时要有特效，光标变锤子」	AI 精准执行

> 现场用第三种 Prompt 让 Codex 改造 Demo 1 的游戏

Demo 3 — AGENTS.md

让 AI 记住项目规矩

延续 Demo 1 的游戏，建立 `AGENTS.md`：

```
# 打地鼠游戏

## 技术栈
HTML + CSS + vanilla JavaScript，不用任何框架

## 规则
- 所有文字使用简体中文
- CSS 使用卡通风格，圆角 + 明亮配色
- 每次修改后要能直接打开 index.html 测试
```

> 之后每次跟 Codex 对话，它通常会先读取并尽量遵守这些规矩

Demo 4 — Skills

让 AI 按照流程做事

```
在项目中创建一个 `code-review` Skill：
每次要做收尾检查时，执行：
1. 检查有没有 console.log 残留
2. 确认所有文字都是简体中文
3. 测试游戏能不能正常打开
4. 列出这次改了哪些文件
```

> 之后你可以明确叫它跑这个 skill，或在任务吻合时让 AI 套用这个流程

Demo 5 — Superpowers

整套技能包

1. 安装 Superpowers
2. 直接跟 Codex 描述任务，必要时明确指定要用的 skill
   例：「请用 brainstorming 先收敛需求，再帮我加多人即时对战模式」
3. 观察它如何把工作拆成比较结构化的流程

观察重点

- 不用自己一个个重写常见 workflow
- AI 会依照安装好的技能包，用比较稳定的方式做事

Demo 6 — OpenSpec

用 Spec 驱动开发

1. 安装 OpenSpec，执行 `openspec init`
2. `/opsx:propose` — 建立需求提案
   （例：「`/opsx:propose` 帮打地鼠游戏加难度选择」）
3. `/opsx:apply` — 确认 spec 后让 AI 实作
4. `/opsx:archive` — 验收完成后归档

> 需求有记录、变更有追踪，AI 照 spec 做事而非靠模糊记忆

Demo 回顾 — 从随便讲讲到有规矩有记录

Demo	主题	你学到的
1	Vibe Coding	一句话做出来
2	Prompts	说清楚，做更好
3	AGENTS.md	规矩写一次，永远生效
4	Skills	流程自动化
5	Superpowers	整套技能包一键安装
6	OpenSpec	需求变成可追踪文档

Part 3: 练习时间 (20 min)

把今天的方法套到你自己的工作

⚠️ 练习前提醒：不要贴机密数据

- **不要贴** API key、密码、token、客户个人信息
- **不要贴** 未公开的核心算法或商业策略
- 不确定能不能贴？→ 先用公开或模拟数据练习

> 详细说明放在回顾段落，这里先记住：**有疑虑就别贴**

自由实作

现在换你把刚刚的做法用在自己的工作上

- 不求做大，只求做完一个可验收的小成果
- 重点不是一次做对，而是让 AI 先做第一版，再由你迭代
- 今天的成功标准：**你知道怎么开始、怎么追问、怎么验收**

自由实作怎么选题目

三个判准

- **小** — 单一功能、单一 bug、单一整理任务，不跨很多系统
- **清楚** — 你能用一句话说清楚要它做什么
- **可验收** — 最后能用测试、界面、或 checklist 判断结果

不适合的题目

- 「帮我重构整个后台」
- 「把整个系统改漂亮」
- 「一次做完整个新产品」

不知道怎么开口？先用这个模板

工程师

「这是我的小任务：____。请你先做第一版，如果需要修改代码就直接改，最后告诉我怎么验收，或帮我跑相关测试。」

QA

「这是我要测的功能 / 这是我观察到的问题：____。请帮我整理成 test checklist 或 bug report，格式要清楚、可直接拿去用。」

BA

「这是我的需求描述：____。请帮我整理成 user story、acceptance criteria，并指出哪里还不够清楚。」

卡住时怎么接着问

常用追问

- 「测试失败，错误如下，请修到通过：____」
- 「结果跟我预期不同，差异是：____」
- 「先不要一次做完，请拆成 3 个步骤」
- 「先告诉我你准备怎么改，再开始动手」

完成标准

- [ ] AI 帮你产出第一版
- [ ] 你有给至少一次反馈或修正方向
- [ ] 最后有验收依据，不是只看感觉

Part 4: 回顾

今天走过的全景地图

```
✅ Vibe Coding   — 先做出来，再逐步修正
✅ Prompts       — 问得越清楚，答得越接近
✅ AGENTS.md     — 让 AI 知道你的项目规矩
✅ Skills        — 让 AI 按固定流程工作
✅ Superpowers   — 把常见流程包成可重用能力
✅ OpenSpec      — 把需求从对话变成可追踪 spec
```

安全提醒：AI 不是保险箱

不要贴给 AI 的东西

类型	示例	风险
机密代码	核心算法、未公开功能	把敏感信息送进第三方服务
客户数据	个人信息、交易记录	违反隐私法规
认证信息	API key、密码、token	泄漏即可被利用
内部文档	未公开的商业策略	竞争信息泄漏

> 是否可使用，依公司政策、工具设置与数据分级规范为准

离开教室后你可以怎么开始

回去就做这三件事

1. 找一个 **30 分钟内能验收** 的小任务
2. 用 AI 先做第一版，不要一开始就追求完美
3. 把错误、差异、限制继续丢回去迭代

三个 Takeaway

1. AI 是你的 Pair Programmer

AI 是协作者，不是代做者；**你要负责验收**

2. AI 不只靠 Prompts

不是越长越好，而是越清楚越好；先讲 **目标、限制、验收**

3. AI 会犯错

卡住时不要自己硬修；把 **错误和预期差异** 丢回 AI

Q&A